

Choosing a Loss Function for Gradient Boosting Regression

A Robustness Case Study

Maaz Babar

Code and dataset: <https://github.com/maazbabar19/Machine-Learning-Tutorial>

Introduction

Most gradient boosting introductions end with the following steps: make trees, fit residuals, end. This is the selection of a loss function that is more important than it seems, and what is typically overlooked. In this tutorial, we consider one particular question in the face of corrupted or unusual values in the training data: When the amount of corrupted or unusual values in the training data is small, does the loss function that is selected for Gradient Boosting Regression actually alter the ability of the model to recover the true relationship in the data? Three loss functions, the standard squared-error loss, Huber loss and quantile (median) loss, are compared directly on the same dataset using the GradientBoostingRegressor from scikit-learn. The outcome isn't subtle. The difference in error between the actual underlying relationship and the choice of loss function is about 2 points for this dataset and is robust across repeated train/test splits and a broad hyperspace.

1. A dataset designed to test this properly

This tutorial requires a dataset to have two characteristics that few real public datasets claim to have: a truly curved relationship, and a known, controllable amount of contamination. Thus, the data set employed here is synthetic, meaning that the daily energy consumption of a commercial office building has been synthetically generated by a known process, and that no reused data sets exist from any tutorial or public repository that was used.

Consumption is not linear with the outdoor temperature, with heating predominating on cold days, cooling predominating on hot days, and consumption being lowest for a temperature, usually near 18°C, which is considered as a comfort temperature. There are also secondary features such as occupancy rate (which is one of the reasons for the higher energy use during the day, and lower at weekends), humidity, and solar irradiance; and intentionally anomalous data points around 5% (with two arbitrary events: equipment problems that typically double or more than double energy use for a day, and building closures or holidays, where energy use drops to a $\frac{1}{4}$ or less of normal). Another 'feature' of the generating process which is known exactly is that the noise-free 'true' energy value is available for each row; this is not true for a real dataset, but is exactly what is needed for testing if a model has learned the real pattern or just memorized the noisy one that it was fed.

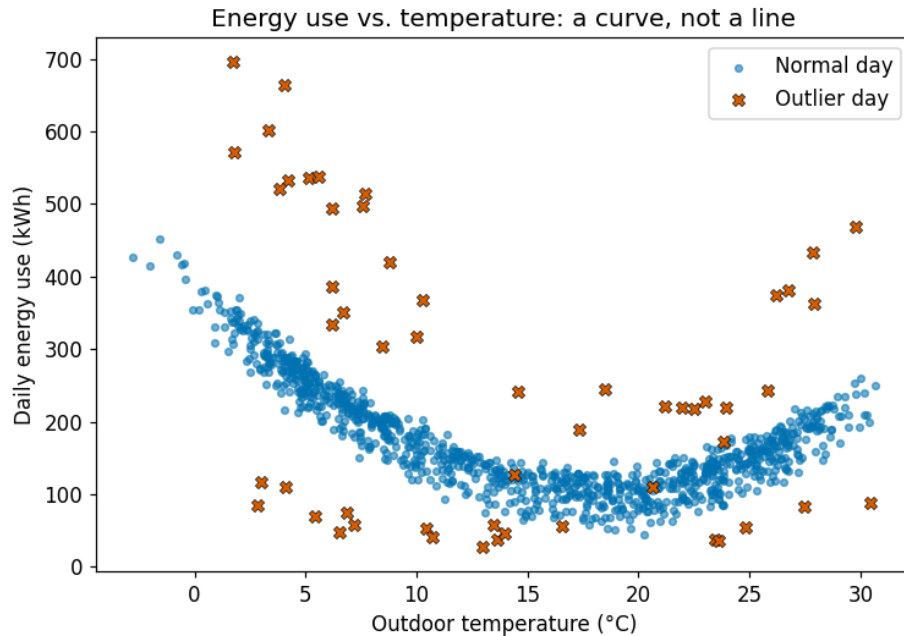


Figure 1. Energy consumption as function of outside temperature. It is not a true linear relationship, rather a curve and there is a small number of outlier days that are well off the curve.

The curve of Figure 1 is clearly curved, and the outliers in both directions are clearly away from the curve, just what is required for meaningful comparisons of loss functions.

This curve should be checked to see whether it is an impression from eye-balling a scatter plot. When all features are compared, the correlation number for outdoor temperature is only about -0.56 , which is less than the actual value, since if a formula is curved, a linear correlation coefficient cannot report it, as the curve could be in either direction from a given comfort point. Solar irradiance, which is closely related to the temperature, has a similar seasonal variation but is not as well correlated with individual occupancy and humidity with regard to energy use.

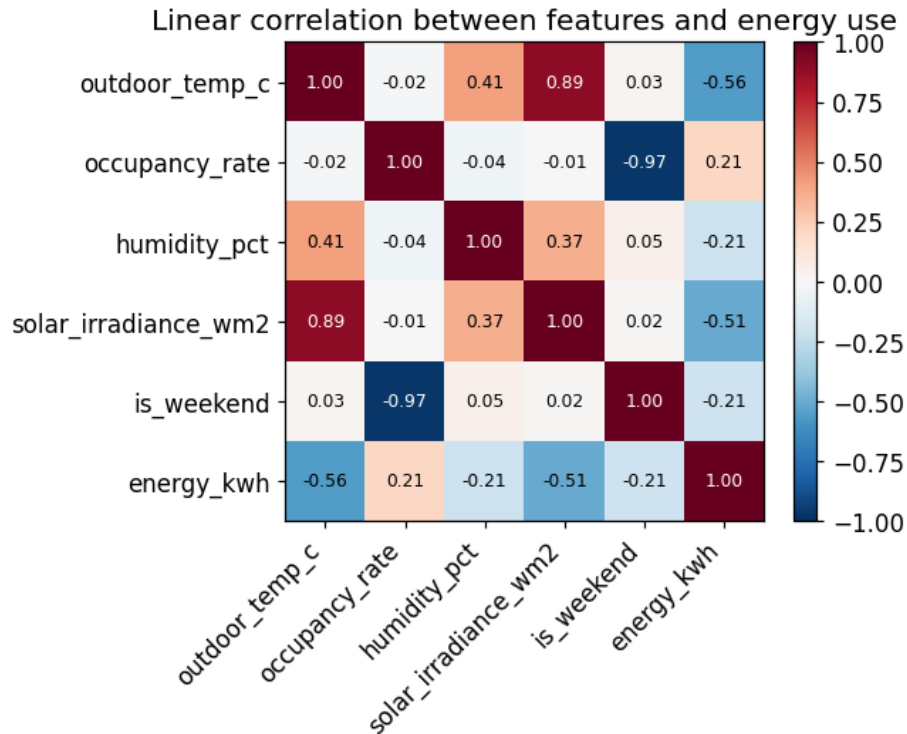


Figure 2. Formal linear correlation of outdoor temperature, occupancy rate, humidity, solar irradiance, weekend status and daily energy use.

The distribution of energy use is also informative in regard to the shape of it. Most days fall between 100 and 250 kWh, but the long right tail in Figure 3 is much higher, with most of the days in this tail being days on which the equipment malfunctioned, described above.

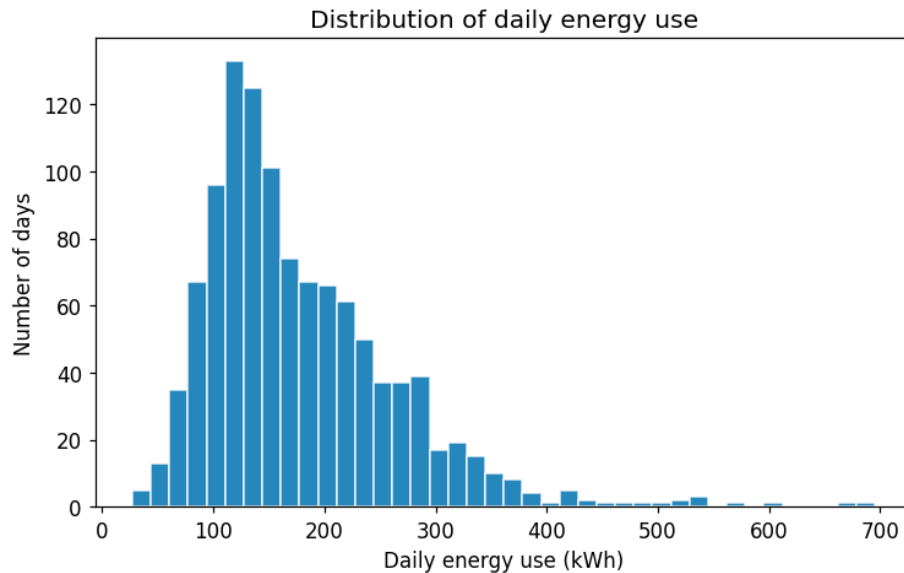


Figure 3. Daily energy consumption distribution for the whole dataset.

2. Why the loss function matters

Before introducing the boosting models, let's first verify that this curve is truly the type that a simple linear model is not capable of modeling. Ordinary Linear Regression, fit on the same training data, is only doing an R^2 of 0.493 vs the true signal, as Figure 4 illustrates: predictions do not follow the diagonal, but rather are capped around 270 kWh no matter how extreme the actual value is, since a single straight line can only run one way in a relationship that bends in two.

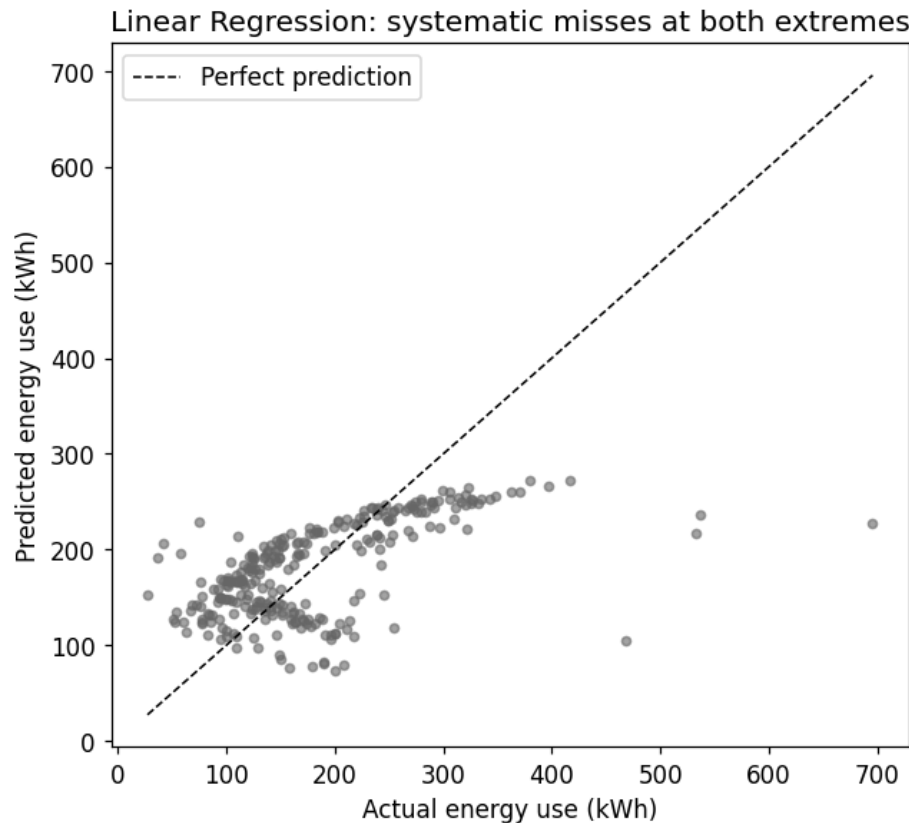


Figure 4. The predictions by linear regression against the actual underlying energy use. The prediction levels do not follow the diagonal at the extremes.

Gradient boosting improves the model by adding one decision tree at a time, where the tree added to the model at each iteration must learn to correct an error made by the previous trees (Friedman, 2001). However, what is considered an “error” is dependent completely on the loss function used for minimization:

- Squared-error loss is to penalize a residual by its square. A day off by 40kWh has a loss impact to sixteen times a day off by 10kWh.
- The Huber loss is similar, except that for smaller residuals it behaves as Huber loss, and, starting at some threshold, which is chosen automatically by scikit-learn at each boosting round, it becomes a straight-line loss. This threshold is called the alpha-quantile of the residuals, where the residuals are the losses that would be obtained from a linear loss function.
- The quantile loss uses the median instead of the mean, and the penalty is linear in the size of the residuals in both directions everywhere (Koenker & Bassett, 1978); here we set $\alpha = 0.5$.

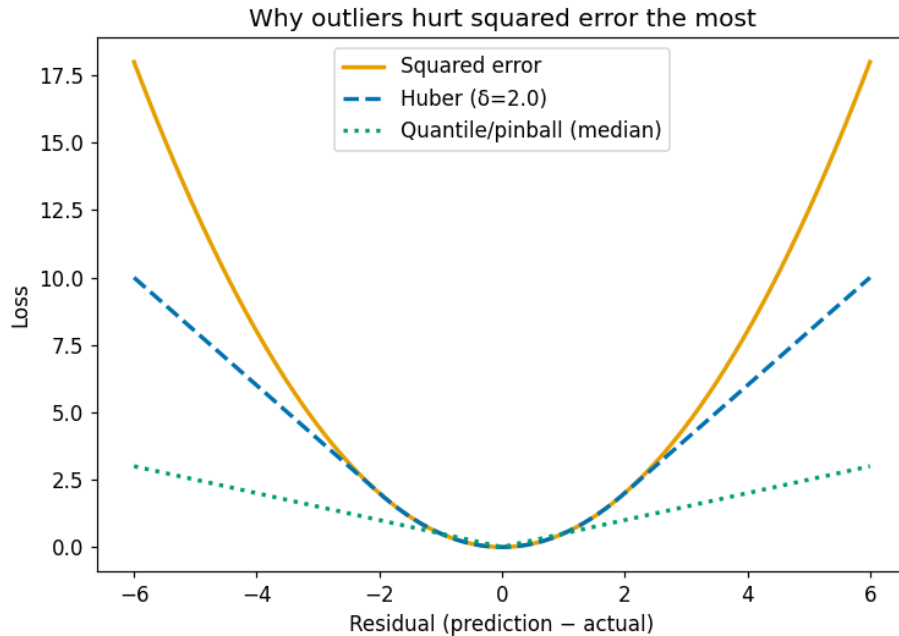


Figure 5. Squared error, Huber and quantile (pinball) loss with the same residual range.

When the residuals are close to zero, the three curves in Figure 5 are virtually indistinguishable, the loss function is not very significant for well-behaved points. The difference is only significant at the extremes: a residual of six results in approximately eighteen units of loss under squared error, ten units under Huber error and three units under quantile loss. A few large errors (known as the "residuals") in the training data can creep up into a squared-error model without causing much harm to the other data, because each of the trees in a boosted ensemble is trying to minimize whatever loss its being used to construct at the time.

3. Results

Three GradientBoostingRegressor models were trained and got the same number of trees (300), max depth (3) and learning rate (0.05) with the same contaminated labels. The models were evaluated on two score sets, one on the observed (contaminated) test labels, which is the only score set available when using real data, and one on the true (noise free) signal, which is only available for these synthetic data.

Model	MAE vs. observed	MAE vs. true signal	R ² vs. true signal
Linear Regression	52.98	45.63	0.493
Gradient Boosting, squared error	22.89	11.14	0.926
Gradient Boosting, Huber ($\alpha=0.9$)	18.60	5.96	0.984
Gradient Boosting, Quantile (median)	18.88	6.38	0.984

As clear as it might be expected to be, all three boosting variants clearly outperform linear regression, as there is a curve relationship between them. The more informative comparison is to Huber loss; Huber loss almost halves the error against the true signal while a quantile loss performs almost identically to Huber. All three models were trained on the same data - different statistics are fed to the trees.

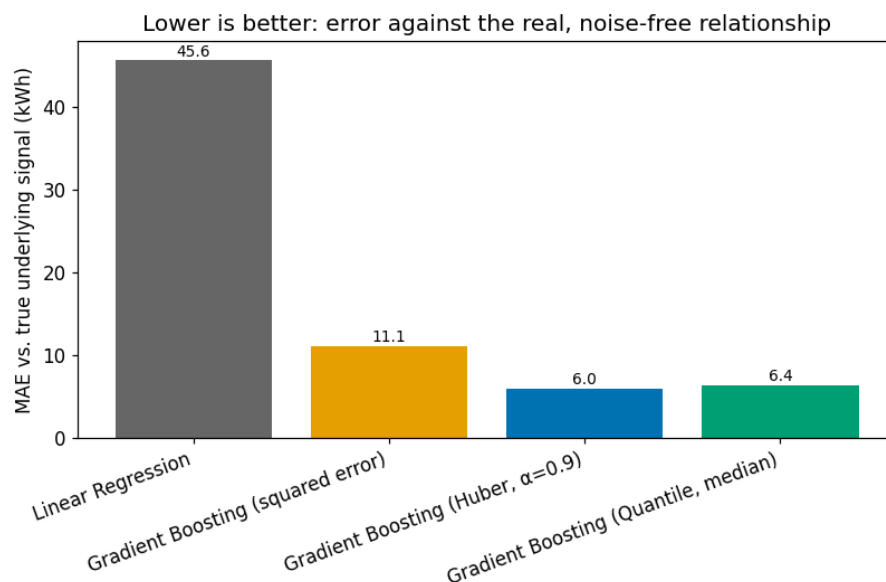


Figure 6. Mean absolute error to the true underlying signal, for each model. Lower is better.

When the predictions of squared-error and Huber are plotted side by side, with the true signal, the mechanism is clearly visible.

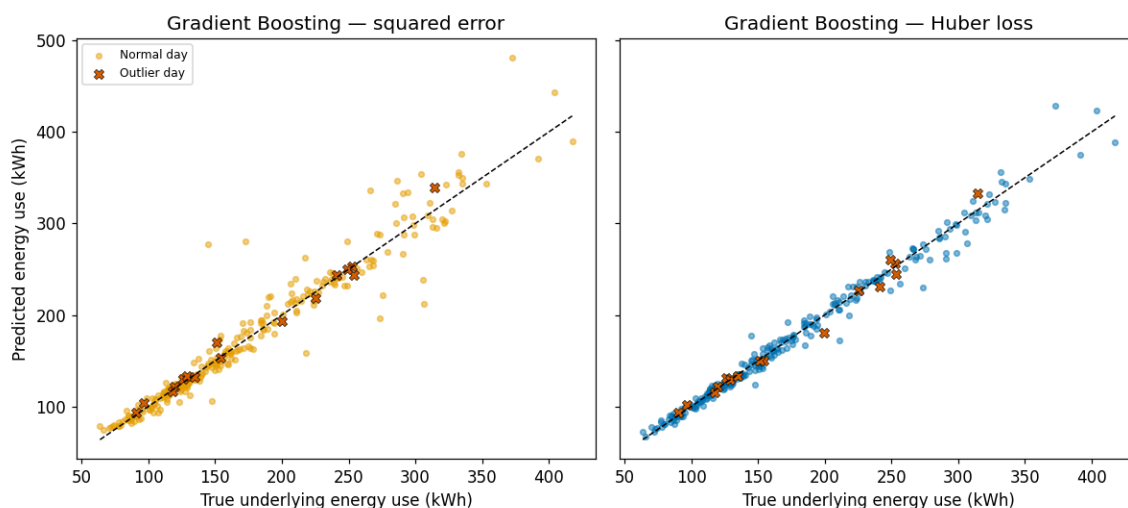


Figure 7. Left shows squared-error and right shows Huber prediction of vs. true use of energy. Outliers are represented by an 'X' in the graph.

A closer look at the Huber panel in Figure 7 reveals that the possible outliers are not alone; there are also points that have no connection to the outliers themselves that are pulled off-target by contamination in the training set, which shows that the contamination in the training set is not only

affecting the squared-error prediction on the contaminated rows. This was by no means an isolated incident. When the comparison was repeated in eight different train/test splits, Huber loss again had lower error compared to the true signal by a fairly similar margin in all cases (Figure 8).

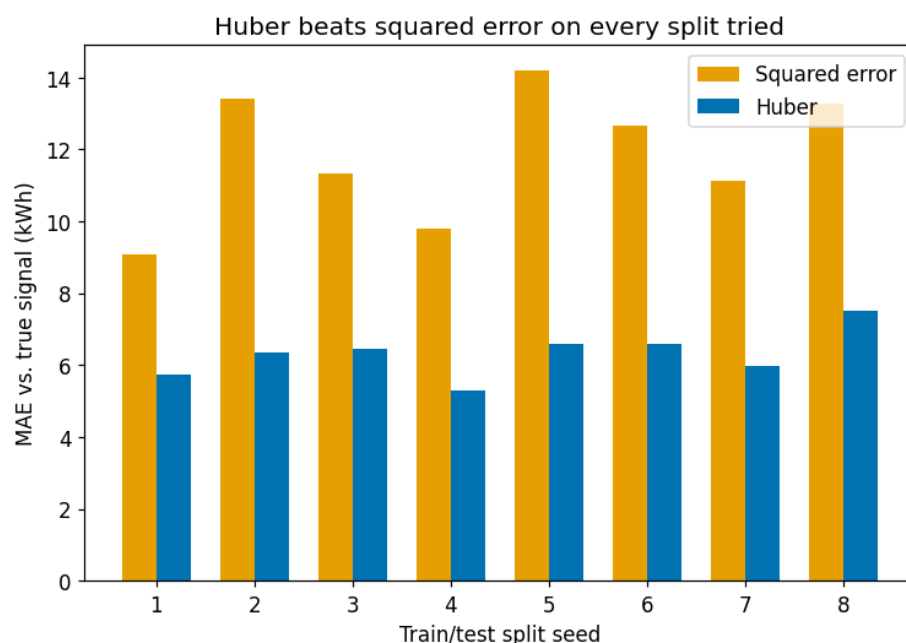


Figure 8. The MAE (Mean absolute error) to the actual signal for squared-error and Huber loss models with 8 different train/test splits.

This discrepancy did not change with number of boosting rounds (between 25 and 500). As fitted with a few more trees (more rounds), both loss functions increase until about 100-150 trees (Figure 9), and then go up when the new rounds fit the outliers more closely than the signal.

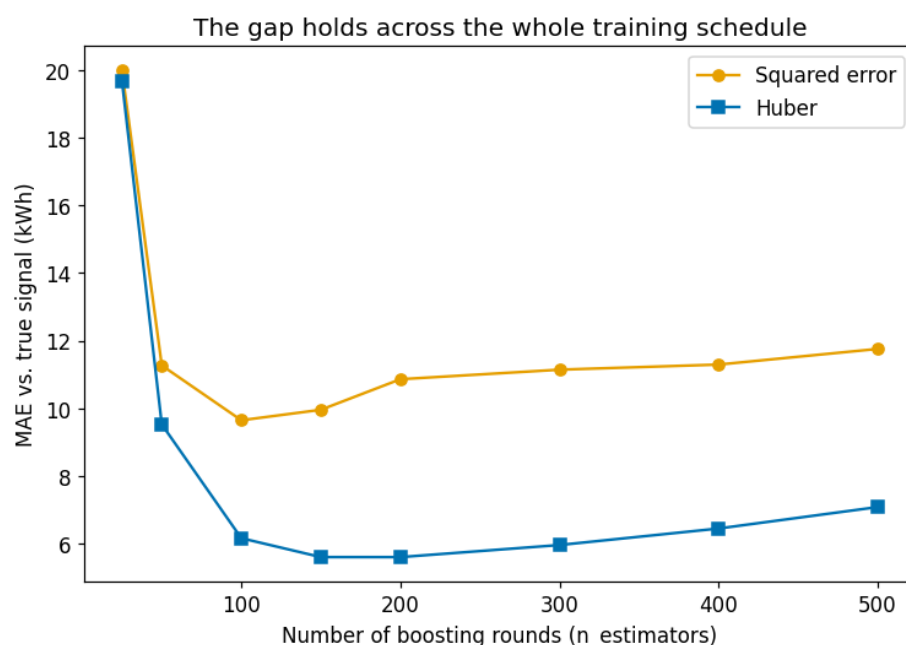


Figure 9. The MAE (squared error and Huber loss) to the true signal as the number of boosting rounds is increased.

In practice, this overfitting bump is easily avoided: The Huber model was able to use as many as 1000 trees, the built-in early stopping of scikit-learn with `validation_fraction=0.15` and `n_iter_no_change=15`, kept the model within 2 trees and no meaningful loss of accuracy (MAE = 6.28 kWh which is the true signal), without the need for any manual search over `n_estimators`.

4. What the model actually learned

There are two additional tests that will help ensure that the model is attempting to do something meaningful and not just getting a higher score. The importances of the features in the Huber loss model put the data in perspective: Outdoor temperature is by far the most important, followed by occupancy, and most other features are virtually insignificant, with respect to the built-in data.

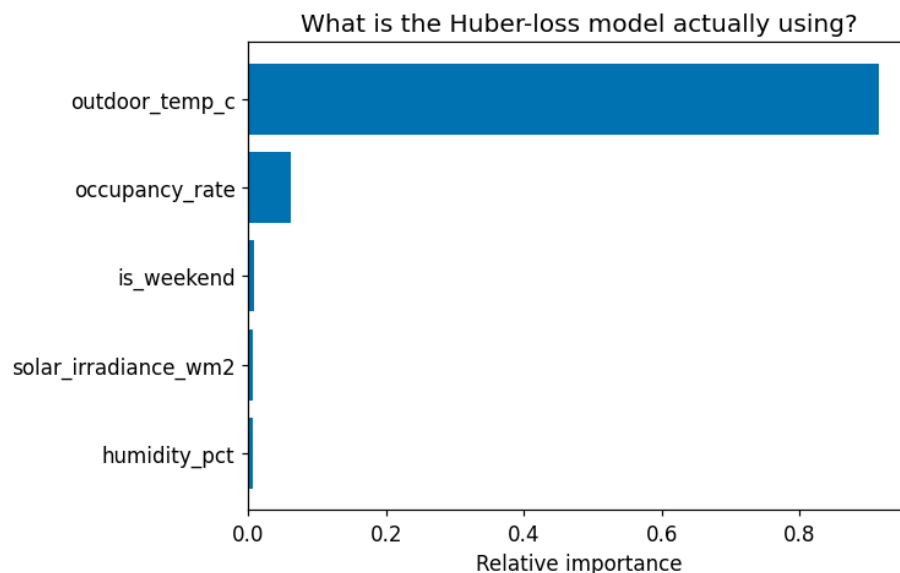


Figure 10. Relative feature importance of Huber-loss gradient boosting model.

A partial dependence plot further steps, and provides a way to isolate the learned relationship between the energy use and the temperature.

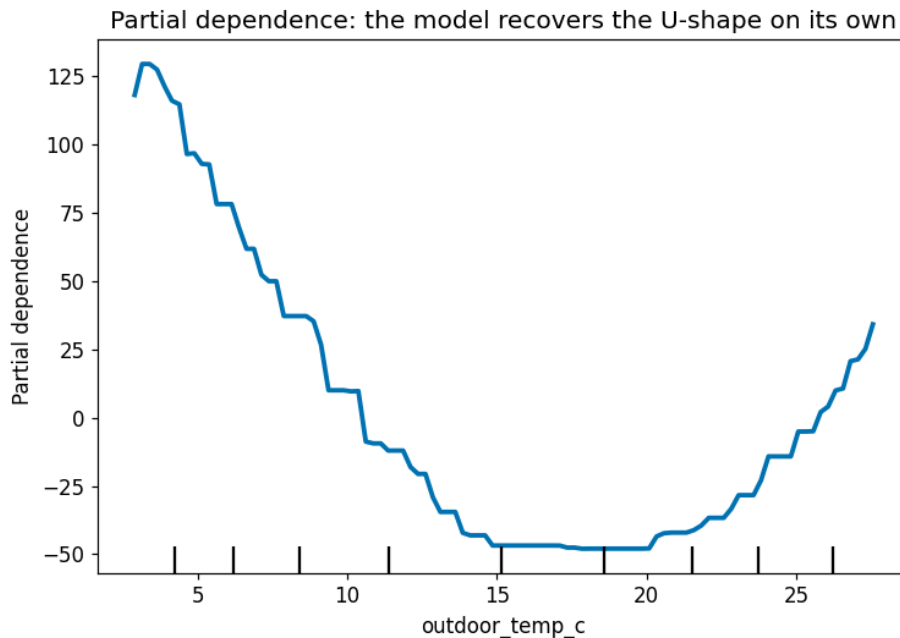


Figure 11. This is a partial dependence of the predicted energy use on the outdoor temperature with Huber-loss model.

The location for a 'comfort point' was never specified for the model. It managed to retrieve a U-shaped curve with a minimum in the high teens which is close to the value programmed into the data (18°C): It only did this due to the pattern that is found in the data. A partial dependence plot that can be physically explained is a reasonable sanity check for any model built on a non-linear relationship that is assumed to be structured: a model that will only capture noise in the data should not have a partial dependence plot that can be physically explained.

5. Takeaways

This is just one set of data and the headline result provides generalized information. A good amount of real tabular data always has some contamination, some of which may be due to sensor faults, data entry errors or events that don't happen regularly, or simply that someone looked for it. The loss function that is most vulnerable to this kind of contamination is the squared-error loss, which is used in most implementations of gradient boosting. The loss function used by most gradient boosting implementations – squared-error loss – is the most exposed one to this type of contamination, since it places larger weights on large residuals. The simple change in the keyword argument, `loss='huber'` or `loss='quantile'`, in scikit-learn, has approximately halved the error of the model from the real underlying relationship in this study.

But robustness introduces trade-offs and is not a free upgrade. The selection of a loss function deliberately not sensitive to large residuals will also be less sensitive to large residuals that are really not mistakes, but rather contaminations, so that the proper setting is still dependent upon which type of mistake is more expensive for the problem in question. The more general lesson to learn from this case study is the easiest to state: Do not blindly believe a regression model on

data that you haven't personally cleaned but instead investigate what it's doing with any rows that it doesn't like.

References

1. **Friedman, J. H. (2001).** Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
<https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full>
2. **Huber, P. J. (1964).** Robust estimation of a location parameter. *Annals of Mathematical Statistics*, 35(1), 73–101.
<https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-35/issue-1/Robust-Estimation-of-a-Location-Parameter/10.1214/aoms/1177703732.full>
3. **Koenker, R., & Bassett, G. (1978).** Regression quantiles. *Econometrica*, 46(1), 33–50.
<https://www.jstor.org/stable/1913643>
4. **Natekin, A., & Knoll, A. (2013).** Gradient boosting machines, a tutorial. *Frontiers in Neuroinformatics*, 7, Article 21.
<https://www.frontiersin.org/journals/neuroinformatics/articles/10.3389/fnbot.2013.00021/full>
5. **Pedregosa, F., et al. (2011).** Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
<https://www.jmlr.org/papers/v12/pedregosa11a.html>